Name: Hasini Kotha
Reg No: 24011101057
Class: AI-DS A

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1
Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Source Code

Google Colab Notebook:

`https://colab.research.google.com/drive/1FkWNyt7JhQYDpUTc7-uIInVzhY7qL8a-?usp=sharing`
`https://colab.research.google.com/drive/125kJQkQ0NYj8i82VncySdVSTGVK4Usrv?usp=sharing`

GitHub Repository:

`https://github.com/Hasini-Kotha/Deep_Learning`

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T\mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	**Output Range**	**Typical Applications**
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset
Source: UCI Machine Learning Repository
Download Link:
`https://archive.ics.uci.edu/dataset/267/banknote+authentication`

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output
Dataset summary and statistical information.

Task 2: Exploratory Data Analysis
Generate

- Histograms

- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation
Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training
Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation
Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

7. Mandatory Plots

Plot	**Purpose**	**Expected Inference**
Feature Histograms	Distribution Analysis	Identify skewness and outliers.
Correlation Heatmap	Feature Relationship	Observe highly correlated features.
Scatter Plot	Class Distribution	Determine whether classes are approximately linearly separable.
Training Error vs Epoch	Learning Behaviour	Verify convergence of the perceptron.
Weight Evolution	Parameter Analysis	Observe stabilization of learned weights.
Bias Evolution	Parameter Analysis	Understand the role of the bias term.
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN.
Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates.
Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features.

7.1 Feature Histograms

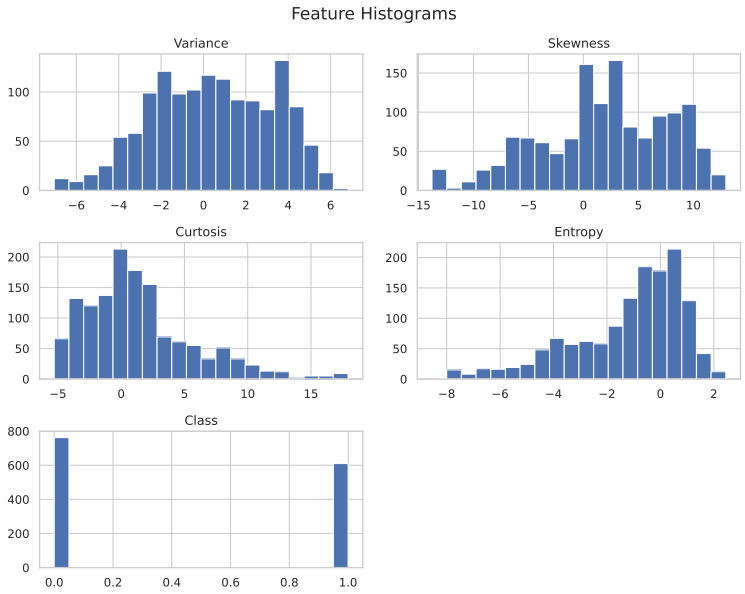


Figure 1: Feature Histograms - Distribution Analysis

Description: The above histograms show that the each feature has a different distribution, with some features spread over a wide range than other hiostograms. A few extreme values in all the features-the outliers are present, but overall the data appears suitable for analysis,the class distribution is balanced, indicacting that both fair notes and duplicate banknotes are there in the dataset.

7.2 Correlation Heatmap

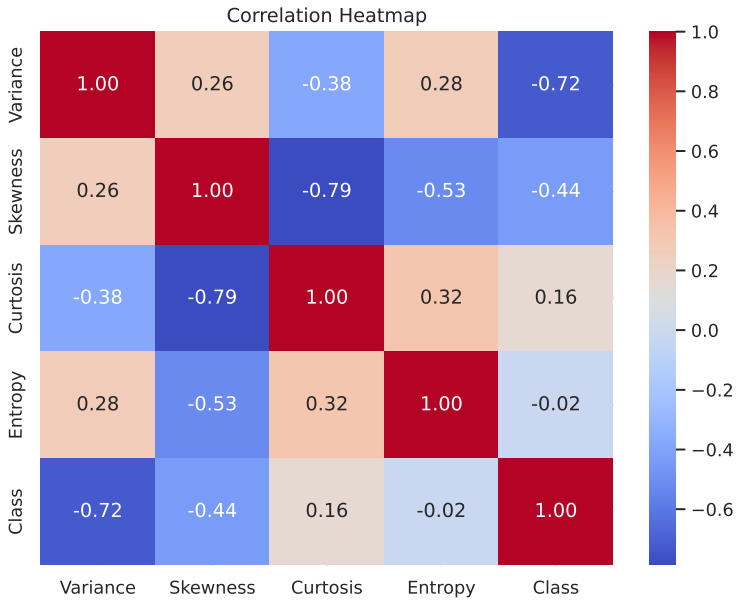


Figure 2: Correlation Heatmap - Feature Relationship

Description: The correlation heatmap shows the strength and direction of features between each one of them. Variance has a strong -ve correlation with the target class (-0.72),making it one of the most influential features for classification, while entropy has almost no correlation with the class (-0.02). Overall, the features have varying levels of correlation, indicating that each contributes different information to the perceptron model where all the features are equally important.

7.3 Scatter Plot

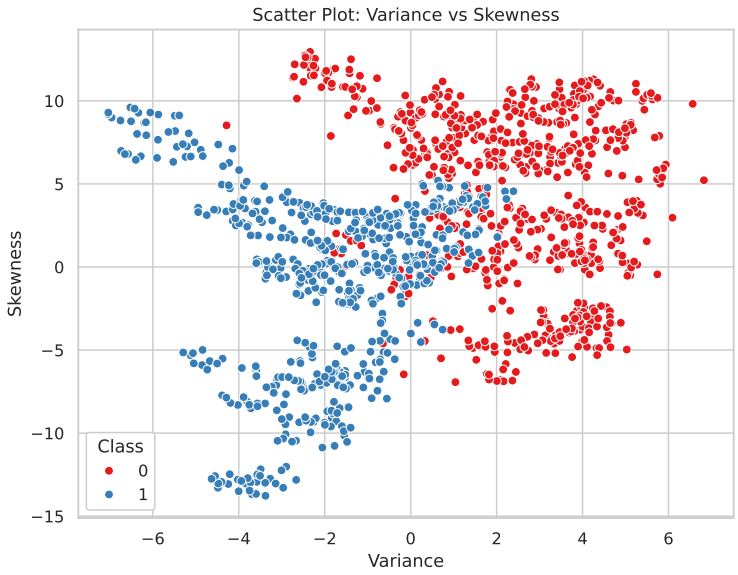


Figure 3: Scatter Plot - Class Distribution

Description: The scatter plot shows that the two classes have a small overlapping regions indicating that the classes are well seperated and the classes are linearly seperable ,making it suitable for classification using single layered perceptron and the features represented in the above picture are variance and skewness .

7.4 Training Error vs Epoch

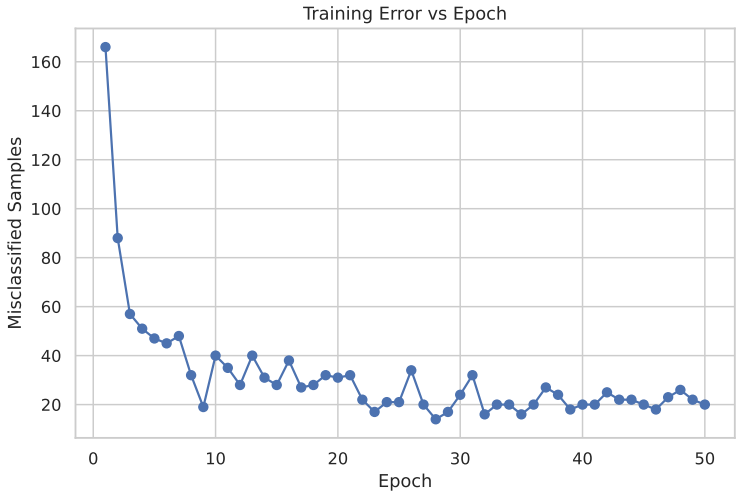


Figure 4: Training Error vs Epoch - Learning Behaviour

Description: The training error decreases significantly during the starting epochs which indicates that the perceptron is learning from the training data and while increase in the epochs the model has reached the stable solution which indicates that there is no improvement in learning and adding more epochs does not produce any improvements.

7.5 Weight Evolution

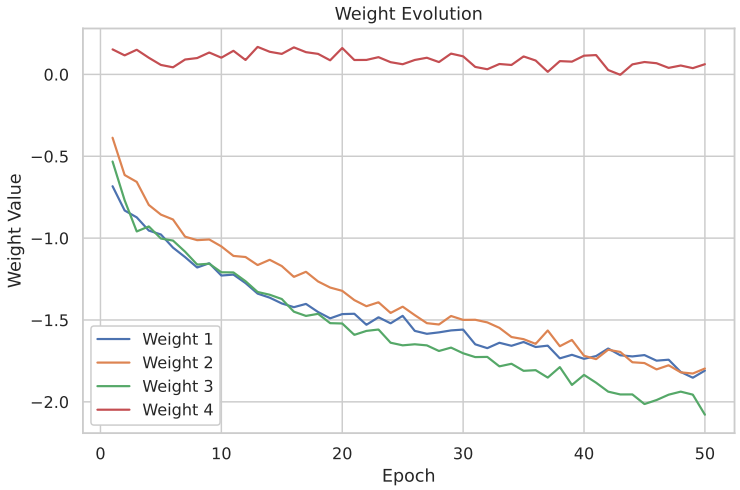


Figure 5: Weight Evolution - Parameter Analysis

Description: The weight values are updated in every epoch based on the perceptron learning algorithm and as training progress ,the changes becomes samaller and the weights become stable which states that the model learned the importance of each feature and reached its convergence.

7.6 Bias Evolution

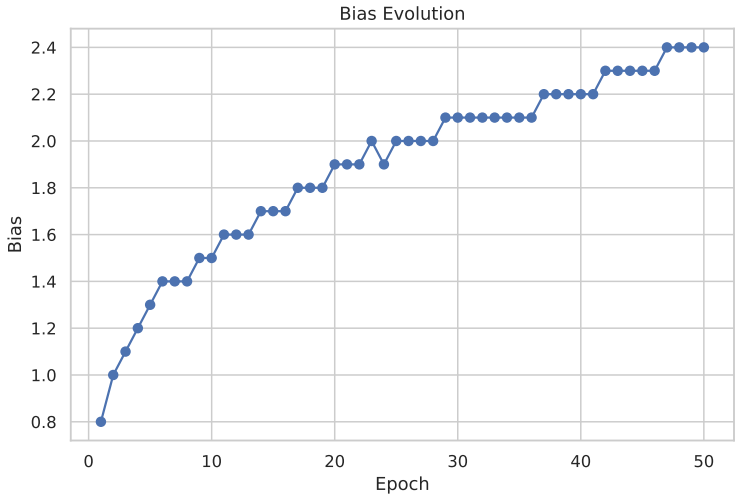


Figure 6: Bias Evolution - Parameter Analysis

Description: The graph shows how the bias is updated during each epoch of training and the gradual stabilization of the bias indicates that the perceotron learned an appropriate decision threshold and the further training results in minimal changes.

7.7 Confusion Matrix

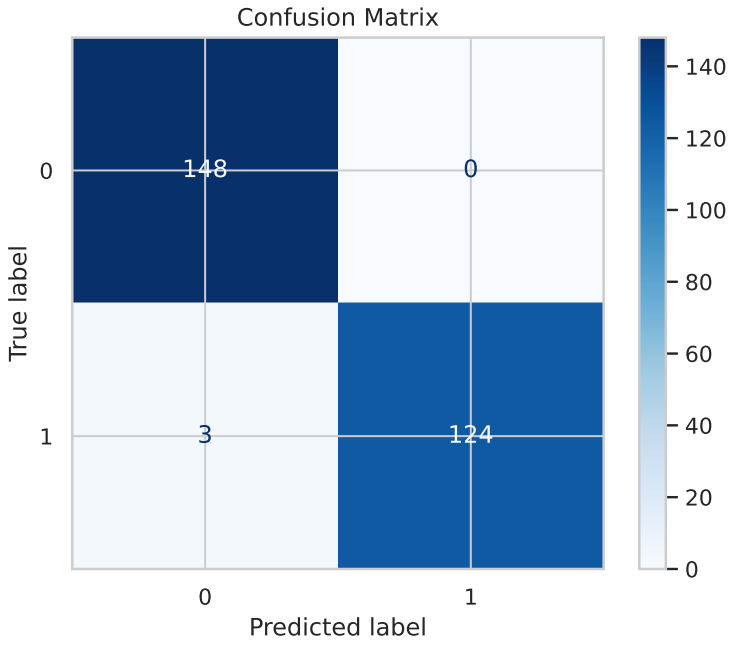


Figure 7: Confusion Matrix - Performance Evaluation

Description: The confusion matrix indicates that the model correctly classified almost all banknotes, with only 3 misclassifications out of 275 test samples. This demonstrates that the perceptron achieved high accuracy and performed well in distinguishing between authentic and forged banknotes.

7.8 Learning Rate Comparison

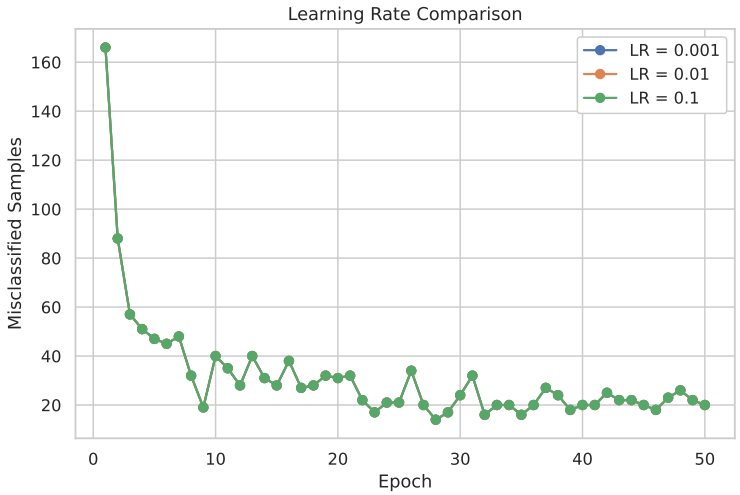


Figure 8: Learning Rate Comparison - Hyperparameter Study

Description: The learning rate comparison shows that all three learning rates produced nearly similar training serror whvih suggests that the change in the learning rate has a very little effect on perceptrons convergence and all the learning rates have achieved similar performance on the dataset.

8. Performance Tables

Training Summary

Metric	Value
Dataset Size	1372 Samples
Train/Test Split	80% Training / 20% Testing (1097 / 275)
Learning Rate	0.01 (Best Performing Learning Rate)
Epochs	50 (Best Training Error at Epoch 28: 14 Misclassified Samples)
Final Weights	[-0.1809, -0.1796, -0.2079, 0.0062]
Final Bias	0.24
Accuracy	98.91%
Precision	100.00%
Recall	97.64%
F1-score	98.80%

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	166	-0.6827	-0.3867	0.8
2	88	-0.8312	-0.6146	1.0
3	57	-0.8732	-0.6566	1.1
4	51	-0.9539	-0.7974	1.2
5	47	-0.9778	-0.8564	1.3
6	45	-1.0581	-0.8862	1.4
7	48	-1.1160	-0.9913	1.4
8	32	-1.1796	-1.0124	1.4
9	19	-1.1535	-1.0082	1.5
10	40	-1.2286	-1.0505	1.5
11	35	-1.2228	-1.1087	1.6
12	28	-1.2748	-1.1154	1.6
13	40	-1.3390	-1.1646	1.6
14	31	-1.3637	-1.1323	1.7
15	28	-1.3991	-1.1707	1.7
16	38	-1.4218	-1.2369	1.7
17	27	-1.4019	-1.2055	1.8
18	28	-1.4513	-1.2647	1.8
19	32	-1.4900	-1.3019	1.8
20	31	-1.4641	-1.3221	1.9
21	32	-1.4620	-1.3786	1.9
22	22	-1.5294	-1.4160	1.9
23	17	-1.4834	-1.3919	2.0
24	21	-1.5205	-1.4572	1.9
25	21	-1.4746	-1.4186	2.0
26	34	-1.5666	-1.4704	2.0
27	20	-1.5845	-1.5194	2.0
28	**14**	**-1.5757**	**-1.5276**	**2.0**
50	20	-1.8092	-1.7965	2.4

9. Conclusion

The experiment successfully demonstrated the implementation of a Single Layer Perceptron from scratch for binary classification using Banknote Authentication dataset,The results proved that the

perceptron with step activation function can effectively classify linearly seperable data with high accuracy . data preprocessing ,including feature normalisation contributed to efficient learning and improved model performance. The trained model achieved an accuracy of 98.91%,along with high precision ,recall,f1-score,indicating reliable classification of authentic and forged banknotes. Overall the experiment provided a clear understanding on perceptron's learning algorithm ,parameter updates,and the role of activation functions in binary classification.

10. Additional Tasks

1. Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.
2. Compare the implementation with Scikit-learn's Perceptron.
3. Repeat the experiment using learning rates 0.001, 0.01 and 0.1.
4. Explain why a Single Layer Perceptron cannot solve the XOR problem.
5. Study the effect of feature normalization on model convergence.

10.1 Comparison of Step and Sigmoid Activation Functions

The Step activation function outputs discrete values like 0 or 1, resulting in a derivative that is zero everywhere except at the threshold where it is undefined. Modern deep learning models rely on gradient-based optimisation algorithms (like backpropgation), which require differentiable activation functions with informative, non-zero gradients to update weights. The Sigmoid function, by contrast, is a smooth, continuous curve that maps inputs to a range between 0 and 1. Its derivative is well-defined everywhere, enabling effective error propgation and weight adjustments in modern deep neural networks.

10.2 Comparison with Scikit-learn's Perceptron

A comparison was made between the custom Perceptron implemented from scratch and the built-in `Perceptron` from Scikit-learn. Both models are trained using the learning rate of 0.01 and 50 epochs.

Metric	**From Scratch**	**Scikit-learn**
Accuracy	98.91%	98.55%
Precision	100.00%	100.00%
Recall	97.64%	96.85%
F1-score	98.80%	98.40%
Final Bias	0.24	0.24

Final Weights:

- **From Scratch:** $[-0.1809, -0.1796, -0.2079, 0.0062]$
- **Scikit-learn:** $[-0.1884, -0.1850, -0.1991, 0.0077]$

Inference: Both models performed very well on the Banknote Authentication dataset,achiveing nearly 98.5% accuracy. The model built from scratch slightly outperformed the scikit-learn model due to internal differences in data shuffling or optimisation techniques used by the library. However, the resulting weights are highly similar and the final bias values are identical (0.24), confirming the correctness and efficiency of the from-scratch implementation.

10.3 Effect of Different Learning Rates

As evidenced by the Learning Rate Comparison plot and training summaries, testing the perceptron with learning rates of 0.001, 0.01, and 0.1 yielded very similar convergence behaviours. Because the Banknote Authentication dataset is largely linearly separable, the perceptron easily identifies a valid decision boundary. Consequently, varying the learning rate within this reasonable range did not drastically alter the number of epochs required to converge or the final accuracy.

10.4 The XOR Problem and Single Layer Perceptrons

A Single Layer Perceptron is fundamentally a linear classifier; it can only draw a single straight line (or hyperplane) to separate classes. The XOR problem represents a non-linearly separable dataset—there is no single straight line that can separate the inputs that yield 1 from those that yield 0. Therefore, a Single Layer Perceptron cannot solve the XOR problem. Complex, non-linear problems like this necessitate multi-layer Perceptrons with hidden layers and non-linear activation functions.

10.5 Effect of Feature Normalization on Convergence

Feature normalization ensures that all input variables are scaled to a similar range. In the Banknote Authentication dataset, features have different numerical scales. Without normalization thefeatures with larger magnitudes would influence weight updates which leads to skewed loss landscape, erratic learning paths and very low and slower convergence. Normalizing the features ensures a more symmetric error surface, which enables the perceptron to converge faster and more stably.

11. Perceptron Learning Algorithm for Logic Gates

11.1 AND Gate

The AND gate was implemented using a Single Layer Perceptron by taking the following truth table as the training dataset.

Table 2: Truth Table for AND Gate

x_1	x_2	Output (y)
0	0	0
0	1	0
1	0	0
1	1	1

The perceptron was initialized with zero weights and zero bias and during the training, the weights and bias were updated whenever a sample was misclassified according to the perceptron learning rule and After every weight update, the corresponding decision boundary was plotted.

Weight Updates and Decision Boundaries

Table 3: Perceptron Weight Updates for AND Gate

Update	Input	Target	Prediction	Weights	Bias
1	$[0, 0]$	0	1	$[0.0, 0.0]$	-0.1
2	$[1, 1]$	1	0	$[0.1, 0.1]$	0.0
3	$[0, 0]$	0	1	$[0.1, 0.1]$	-0.1
4	$[0, 1]$	0	1	$[0.1, 0.0]$	-0.2
5	$[1, 1]$	1	0	$[0.2, 0.1]$	-0.1
6	$[0, 1]$	0	1	$[0.2, 0.0]$	-0.2
7	$[1, 0]$	0	1	$[0.1, 0.0]$	-0.3
8	$[1, 1]$	1	0	$[0.2, 0.1]$	-0.2

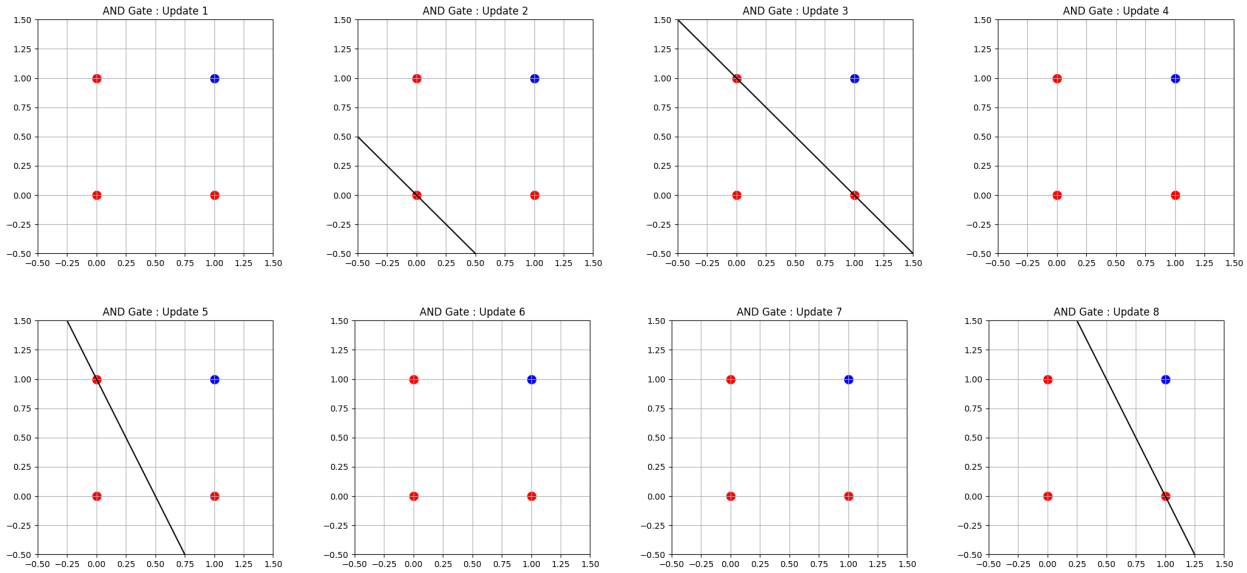


Figure 9: Decision boundaries after each weight update (Updates 1 to 8).

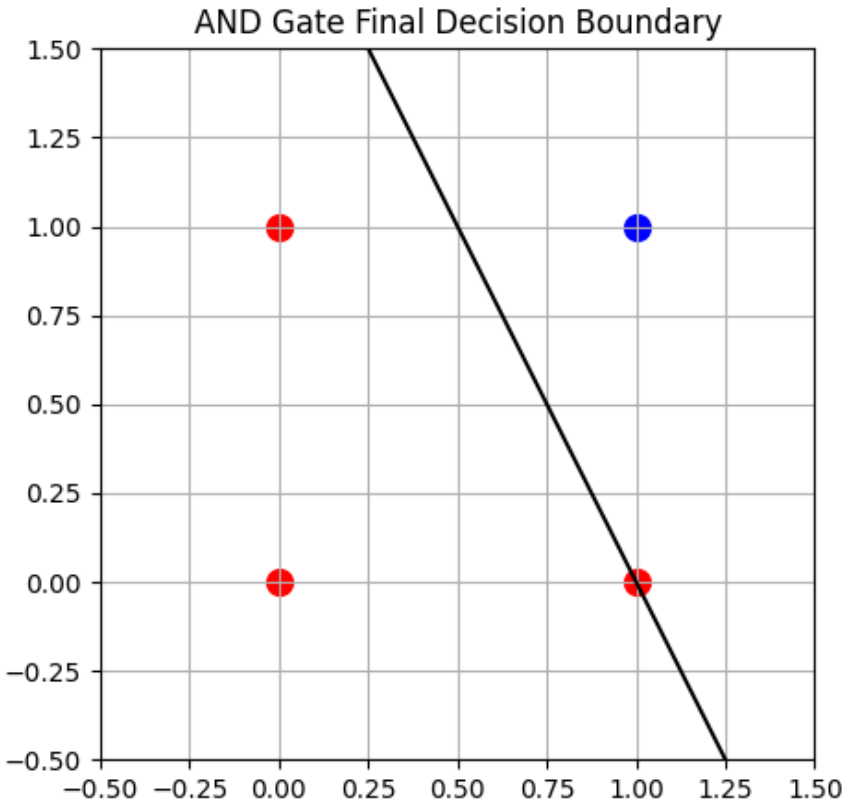
Final Learned Parameters

Final Weights: [0.2, 0.1]
Final Bias: -0.2

Final Predictions

Input	Predicted Output
$(0, 0)$	0
$(0, 1)$	0
$(1, 0)$	0
$(1, 1)$	1

Final Decision Boundary



Observation:

The perceptron successfully learned the AND gate after 8 updates and the final decision boundary correctly seperates the positive class from he remaining input combinations , which demonstartes that the AND gate is linearly seperable and can be implemneted with SLP.

11.2 OR Gate

The OR gate was implemented using a Single Layer Perceptron with the following truth table as the training dataset.

Table 4: Truth Table for OR Gate

x_1	x_2	Output (y)
0	0	0
0	1	1
1	0	1
1	1	1

The perceptron was initialized with zero weights and zero bias and during the training, the weights and bias were updated whenever a sample was misclassified according to the perceptron learning rule and After every weight update, the corresponding decision boundary was plotted.

Weight Updates and Decision Boundaries

Table 5: Perceptron Weight Updates for OR Gate

Update	**Input**	**Target**	**Prediction**	**Weights**	**Bias**
1	$[0, 0]$	0	1	$[0.0, 0.0]$	-0.1
2	$[0, 1]$	1	0	$[0.0, 0.1]$	0.0
3	$[0, 0]$	0	1	$[0.0, 0.1]$	-0.1
4	$[1, 0]$	1	0	$[0.1, 0.1]$	0.0
5	$[0, 0]$	0	1	$[0.1, 0.1]$	-0.1

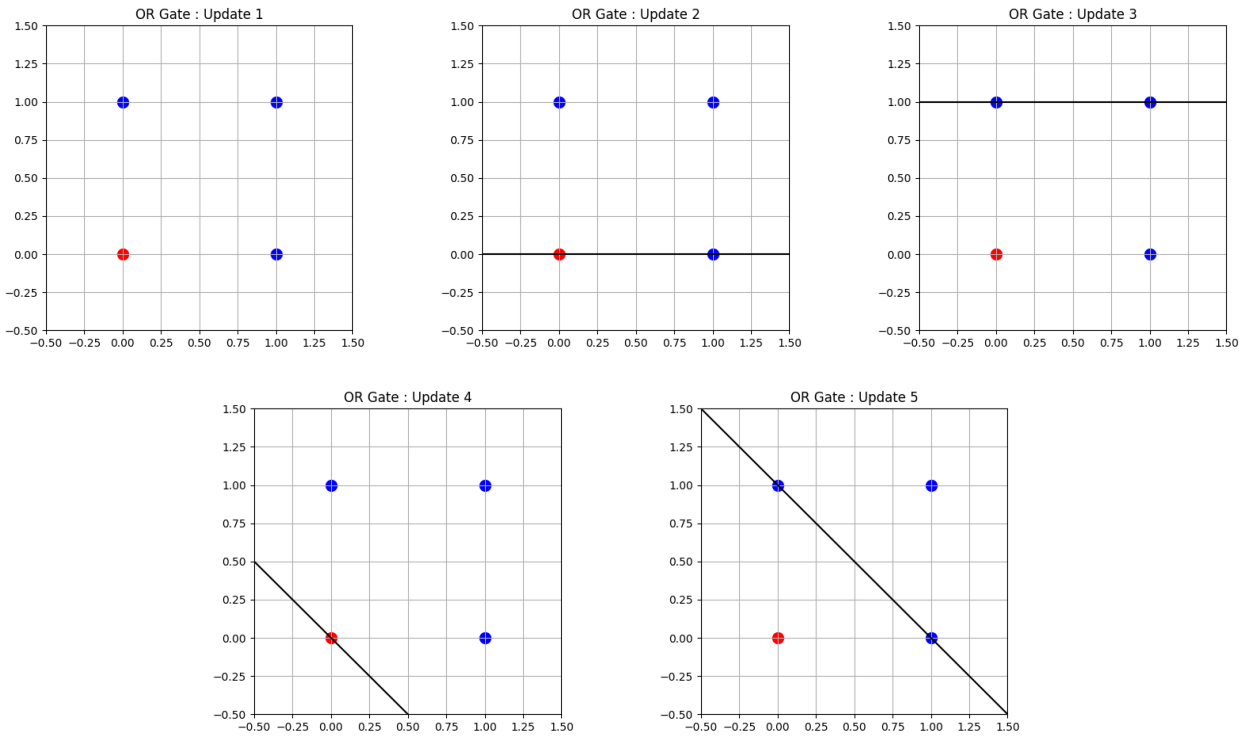


Figure 10: Decision boundaries after each weight update for OR Gate (Updates 1 to 5).

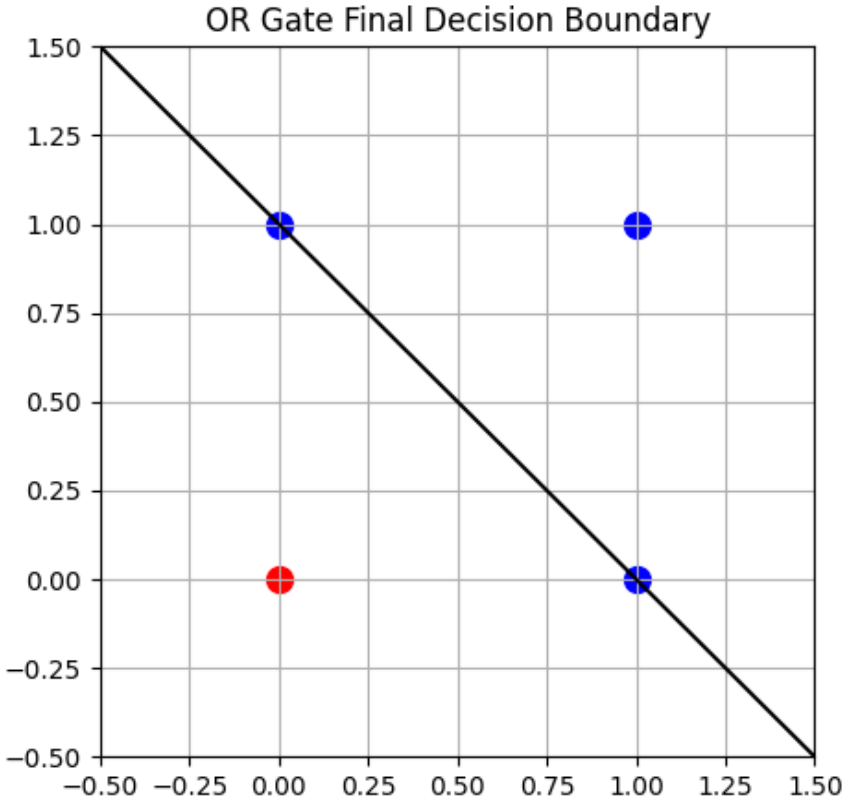
Final Learned Parameters

Final Weights: [0.1, 0.1]
Final Bias: -0.1

Final Predictions

Input	Predicted Output
$(0, 0)$	0
$(0, 1)$	1
$(1, 0)$	1
$(1, 1)$	1

Final Decision Boundary



Observation:

The perceptron successfully learned the OR gate after five weight updates,and the final decision boundary correctly seperates the negative class from positive class inputs which demonstrates that the OR gate is linearly seperable and can be implemented using SLP.

11.3 NOT Gate

The NOT gate was implemented using a Single Layer Perceptron with the following truth table as the training dataset.

Table 6: Truth Table for NOT Gate

x	Output (y)
0	1
1	0

The perceptron was initialized with a zero weight and zero bias. During training, the weight and bias were updated whenever a sample was misclassified according to the perceptron learning rule.

Weight Updates

Table 7: Perceptron Weight Updates for NOT Gate

Update	**Input**	**Target**	**Prediction**	**Weight**	**Bias**
1	1	0	1	-0.1	-0.1
2	0	1	0	-0.1	0.0

Final Learned Parameters

Final Weight: -0.1
Final Bias: 0.0

Final Predictions

Input	Predicted Output
0	1
1	0

Observation:

The perceptron successfully learned the NOT gate after just two weight updates. The simple nature of the NOT gate allowed for rapid convergence, correctly mapping the single input feature to its inverse.

11.4 XOR Gate

The XOR gate was implemented using a Single Layer Perceptron with the following truth table as the training dataset.

Table 8: Truth Table for XOR Gate

x_1	x_2	Output (y)
0	0	0
0	1	1
1	0	1
1	1	0

The perceptron was initialized with zero weights and zero bias. During training, the weights and bias were updated whenever a sample was misclassified. Since the algorithm failed to converge, training was stopped after 10 epochs (38 updates). Below are the initial and final weight updates.

Weight Updates and Decision Boundaries

Table 9: Selected Perceptron Weight Updates for XOR Gate

Update	Input	Target	Prediction	Weights	Bias
1	$[0, 0]$	0	1	$[0.0, 0.0]$	-0.1
2	$[0, 1]$	1	0	$[0.0, 0.1]$	0.0
3	$[1, 1]$	0	1	$[-0.1, 0.0]$	-0.1
4	$[0, 1]$	1	0	$[-0.1, 0.1]$	0.0
... Updates 5 - 34 ...					
35	$[0, 0]$	0	1	$[-0.1, 0.0]$	-0.1
36	$[0, 1]$	1	0	$[-0.1, 0.1]$	0.0
37	$[1, 0]$	1	0	$[0.0, 0.1]$	0.1
38	$[1, 1]$	0	1	$[-0.1, 0.0]$	0.0

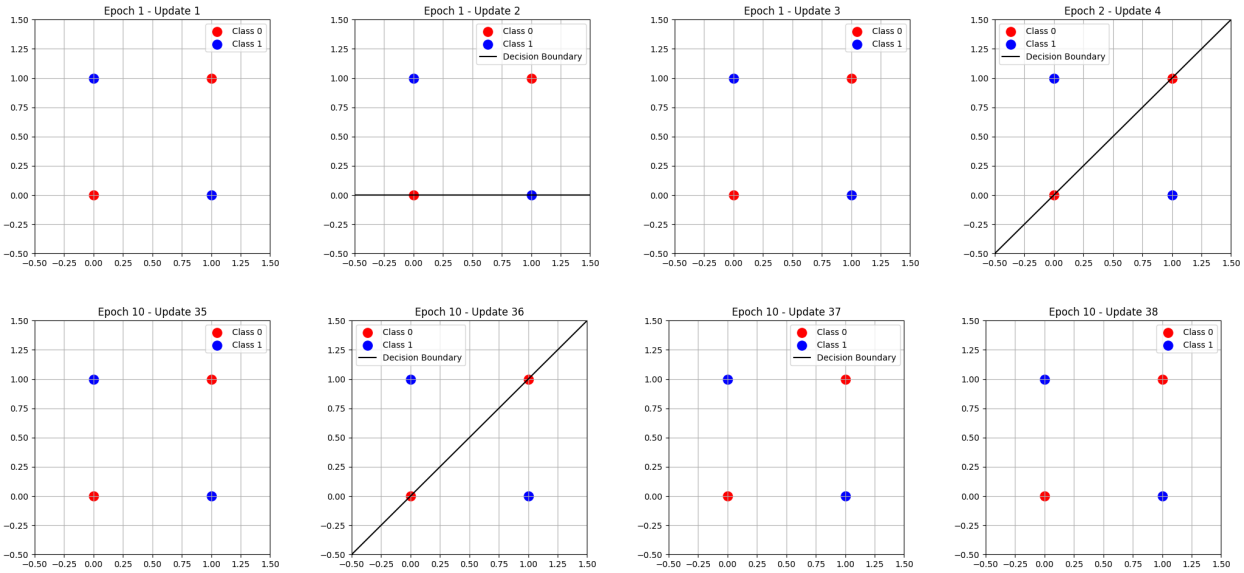


Figure 11: Decision boundaries for the first 4 and last 4 weight updates for XOR Gate.

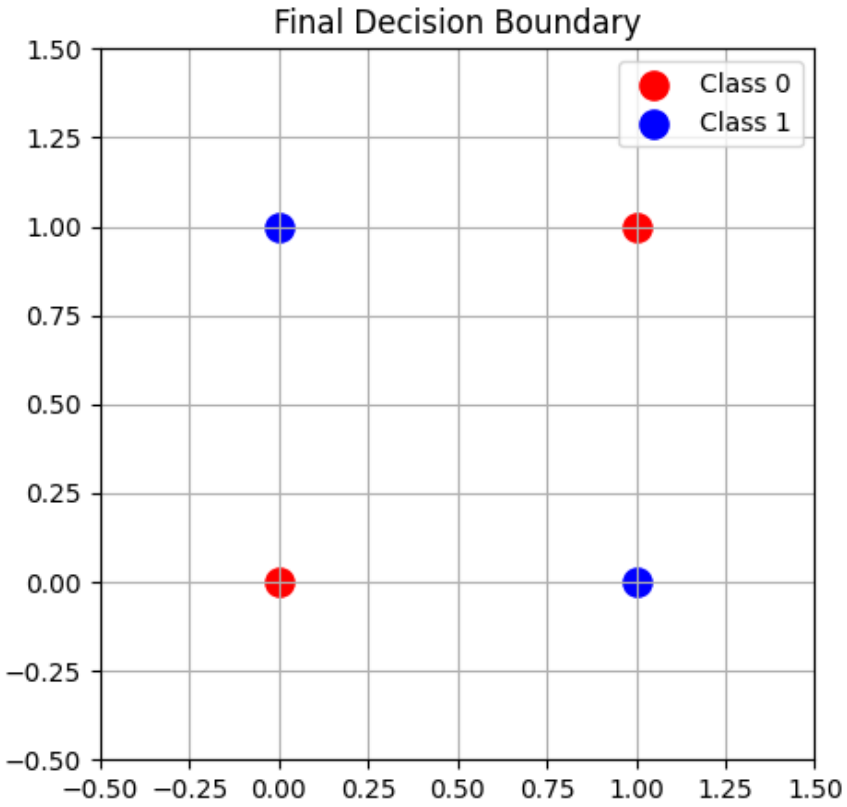
Final Learned Parameters

Final Weights: [-0.1, 0.0]
Final Bias: 0.0

Final Predictions

Input	Predicted Output
$(0, 0)$	1
$(0, 1)$	1
$(1, 0)$	0
$(1, 1)$	0

Final Decision Boundary



Observation:

The perceptron fails to converge for the XOR gate because the XOR dataset is not linearly seperable ,not even a single straight line can seperate the two classes , so the weights keep changing during training and algorithm cannot correctly classify all 4 samples which demonstrates the fundamental limitation of SLP.